Algorithms

Week 1 – Introduction to Algorithms

Korea University Sejong Campus, Department of Computer Science

Part 1. OT

Instructor

Unggi Lee, codingchild@korea.ac.kr

- Assistant Professor, Dept. of Computer Science, Korea University Sejong Campus
- Previously:
  - Assistant Professor, Dept. of Computer Engineering, Chosun University
  - AI/NLP Engineer in Global EdTech (Enuma, I-Scream Edu)
  - Elementary School Teacher
- Research (Google Scholar):
  - AIED: Generative AI in Education, Pedagogical Alignment, Knowledge Tracing
  - NLP & Robotics: Large Language Models (LLMs), Vision-Language-Action (VLA)
- Lab Activities (Lab):
  - Published a VLA preprint with undergraduates at Chosun University
  - Industry-academia partnerships with Upstage, Newdive, and others

Syllabus

- You can find the syllabus in **LMS**
- Total **15 weeks**
  - 8th week – **Midterm Exam**
  - 15th week – **Final Exam**

Grading

Component	Weight
Assignment	**10%**
Midterm Exam (written)	**30%**
Final Exam – Project	**30%**
Final Exam – Written	**30%**
Attendance	0%

> However, absent for **one-third (1/3)** or more of the total class hours -> no grade will be awarded.

Assignment (10%)

In-class Quiz: 5%

- Quizzes in **10 classes** -> each **0.5%**
- Week 03, 04, 05, 06, 07, 09, 10, 11, 12, 13
- Given at the **start** of the 1st hour (~15 min), covering **previous week's content**
- Quiz questions may appear on the **written exams**
- **No GenAI usage** allowed – quizzes assess whether you have internalized the material

Take-home Assignment: 5%

- Assignments in **5 classes** -> each **1%**
- Week 02, 03, 04, 05, 06

Midterm & Final Exam

- Midterm: **30%** – Handwritten (digital devices not allowed), 1 hour
- Final – Written: **30%** – Handwritten, 1 hour
- Final – Project: **30%** – Team project (Weeks 09–13), see project guidelines

Class Format

Hour	Content
1st	Quiz (~15 min) + Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

- Textbook: **Introduction to Algorithms, 3rd Edition** (CLRS)
- Quiz is about **previous week's** content, at the **start** of the 1st hour

Why Study Algorithms?

- **The most important subject** in Computer Science
- Algorithms train you in **how to think** as a programmer
- The **thinking process** behind each algorithm matters more than the algorithm itself

Applications:

- Design of every program
- Formal representation of problems and their solutions
- Analysis of program efficiency and complexity

What We'll Learn

- Various algorithms for various problems
  - Greedy, Dynamic Programming, Divide and Conquer, Graph algorithms, …
- Systematic thinking through algorithmic problem solving
- **Formal representation** of solutions
  - Flowcharts, Pseudocode
- **Efficiency and complexity analysis**
  - How execution time grows with input size

Textbook

Main textbook:

- *Introduction to Algorithms*, 3rd Edition (CLRS)
  - Cormen, Leiserson, Rivest, Stein

Key idea throughout the course:

```
Problem Solving ⟷ Divide & Conquer ⟷ Recursive Thinking ⟷ Recurrence
```

Coding Test Landscape

Algorithms are at the core of technical interviews and coding tests:

Platform	URL
Baekjoon (BOJ)	https://www.acmicpc.net/
Programmers	https://programmers.co.kr/
LeetCode	https://leetcode.com/
Codeforces	https://codeforces.com/
solved.ac	https://solved.ac/

Visualization tools (great for studying):

- VisuAlgo: https://visualgo.net/

Part 2. What is an Algorithm?

Definition, Representation, and Key Concepts

What is an Algorithm?

> A **systematic description** of a problem-solving procedure.

- Given an **input** (the problem), after a **finite number of steps**, it produces the desired **output** (the result).

```
┌──────────┐   ┌──────────────────┐   ┌──────────┐
│  Input   │ → │    Algorithm     │ → │  Output  │
│(Problem) │   │ (finite steps)   │   │ (Result) │
└──────────┘   └──────────────────┘   └──────────┘
```

Requirements:

- Clearly specify input and output
- The algorithm describes the process of transforming input into output

Algorithm Example: Find the Maximum

Problem: Find the maximum value among 100 students' exam scores.

- **Input:** 100 exam scores
- **Output:** The maximum score

```
def find_max(scores):
    max_val = scores[0]
    for i in range(1, len(scores)):
        if max_val < scores[i]:
            max_val = scores[i]
    return max_val
```

This is a simple but complete algorithm – it has a clear input, a finite procedure, and a well-defined output.

Algorithm Example: Sorting

Problem: Sort n integers in ascending order.

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

```
Input:  [64, 25, 12, 22, 11]
Step 1: [11, 25, 12, 22, 64]   ← swap 64 and 11
Step 2: [11, 12, 25, 22, 64]   ← swap 25 and 12
Step 3: [11, 12, 22, 25, 64]   ← swap 25 and 22
Step 4: [11, 12, 22, 25, 64]   ← already in place
Output: [11, 12, 22, 25, 64]
```

Algorithm vs Data Structure

	Data Structure	Algorithm
Question	How to **store** information?	How to **solve** a problem?
Analogy	Parts and modules of a car	The method of building a car
Prerequisites	–	Basic Programming, Data Structures

> **Algorithms + Data Structures = Programs**
> – Niklaus Wirth

Example: English Dictionary

- Unsorted words -> sequential search (slow)
- Sorted words -> binary search (fast)
- The **data structure** (sorted vs unsorted) determines which **algorithm** is efficient

Course Roadmap

Week	Topic	Week	Topic
1	Introduction to Algorithms	**9**	Graph Algorithms
2	Algorithm Analysis (Complexity)	**10**	Shortest Path
3	Divide and Conquer (1)	**11**	Dynamic Programming (1)
4	Divide and Conquer (2)	**12**	Dynamic Programming (2)
5	Greedy Algorithms (1)	**13**	String Matching
6	Greedy Algorithms (2)	**14**	NP-Completeness
7	Sorting & Selection	**15**	*Final Exam*

Part 3. First Steps in Algorithms

Classic Problems and Algorithmic Thinking

Origin of the Word "Algorithm"

- The word **"algorithm"** comes from the 9th-century Persian mathematician **al-Khwarizmi**
- The **first known algorithm**: Euclid's GCD algorithm (~300 BC)

```python
def gcd(a, b):
    while b ≠ 0:
        a, b = b, a % b
    return a

# Example: gcd(48, 18)
# 48, 18 → 18, 12 → 12, 6 → 6, 0 → return 6
```

Algorithms have been around for over **2,300 years** – long before computers existed.

1.1 Finding the Maximum

Problem: Given cards with numbers face down, find the card with the largest number.

Approach:

1. Look at the first card and remember its number
2. Look at the next card – if it's larger, update the remembered number
3. Repeat until all cards are checked
4. The remembered number is the maximum

```
def find_max(cards):
    max_val = cards[0]          # Step 1: remember first card
    for i in range(1, len(cards)):
        if cards[i] > max_val:  # Step 2: compare
            max_val = cards[i]  # update if larger
    return max_val              # Step 4: return the max
```

This is **sequential search** – reading cards one by one in order.

1.2 Finding a Specific Number

Problem: Find the number 85 among these cards.

Sequential Search:

- Remember 85, then check each card one by one
- In sorted data `[15, 20, 25, 35, 45, 55, 60, 75, 85, 90]`, sequential search would take **9 comparisons** to find 85

```
15 → 20 → 25 → 35 → 45 → 55 → 60 → 75 → 85  Found!
 1    2    3    4    5    6    7    8    9
```

Can we do better if the data is sorted?

Binary Search

Key insight: If data is **sorted**, we can use the ordering information.

Approach: Compare with the **middle** element, then search only **one half**.

```
Search for 85 in: [15, 20, 25, 35, 45, 55, 60, 75, 85, 90]

Step 1: middle = 45  →  85 > 45  →  search right half
        [55, 60, 75, 85, 90]

Step 2: middle = 75  →  85 > 75  →  search right half
        [85, 90]

Step 3: middle = 85  →  85 = 85 →  FOUND!
```

Only **3 comparisons** instead of 9!

Binary Search – Algorithm

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left ≤ right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid              # Found!
        elif arr[mid] < target:
            left = mid + 1          # Search right half
        else:
            right = mid - 1         # Search left half

    return -1                       # Not found
```

Comparison:

Method	Sorted data required?	Worst case (n items)
Sequential Search	No	n comparisons

1.3 Coin Change Problem

Problem: Pay 730 won in change using the **fewest coins** possible.

Available coins: 500, 100, 50, 10 won

Greedy approach: Always pick the **largest** coin that doesn't exceed the remaining amount.

```
Remaining: 730
  → 500 won x 1  (remaining: 230)
  → 100 won x 2  (remaining:  30)
  →  10 won x 3  (remaining:   0)

Total coins: 1 + 2 + 3 = 6 coins (minimum!)
```

Greedy Algorithm

> At each step, make the **locally optimal choice** (the "greedy" choice) hoping it leads to a **globally optimal solution**.

Coin change greedy strategy:

- Always select the largest denomination that doesn't exceed the remaining amount

```
def coin_change(amount, coins=[500, 100, 50, 10]):
    result = []
    for coin in coins:                # coins sorted descending
        count = amount // coin
        if count > 0:
            result.append((coin, count))
            amount -= coin * count
    return result

# coin_change(730) → [(500, 1), (100, 2), (10, 3)]
```

Important question: How can we **guarantee** this greedy approach always gives the minimum?

- We'll study this rigorously in **Chapter 4 (Greedy Algorithms)**

1.4 Euler Path (One-Stroke Drawing)

Problem: Starting from a vertex, traverse **every edge exactly once** and return to the starting vertex. Vertices may be revisited.

```
Example graph:

    1 --- 2
    |\ /| |
    | X | |
    |/ \| |
    4 --- 3
```

This is the **Euler Circuit** problem.

Euler Path – The Key Insight

When at vertex 7, should we go to vertex 6, 9, or 10?

The rule: Move to a neighbor only if there is a **cycle** back to the current vertex through that neighbor.

- Going to vertex 6: there exists a cycle back through 5, 4, 3, 9, 7 – **safe**
- Going to vertex 9: there exists a cycle back through 3, 4, 5, 6, 7 – **safe**
- Going to vertex 10: only leads to vertex 1, no cycle back – **dead end**

Algorithm:

1. If there is only one adjacent unvisited edge (a "bridge"), take it
2. Otherwise, move to a neighbor through which a **cycle** exists back to the current vertex

Cycle detection can be done using **Depth First Search (DFS)**.

1.5 Maze Solving

The Greek myth of Theseus:

- Theseus entered the Labyrinth with a **ball of thread**, unwinding it as he went
- He slew the Minotaur and followed the thread back to escape

Without a thread:

Right-Hand Rule:

1. Place your right hand on the wall
2. Walk forward, never lifting your right hand from the wall
3. You will always reach the exit

This works without any markers or threads – a simple, elegant algorithm.

1.6 Fake Coin Problem

Problem: Among n coins, **one** is counterfeit (slightly lighter). Find it using a **balance scale** with the minimum number of weighings.

Three approaches:

Approach	Method	Worst case (n coins)
Approach A	Compare one coin against each other	n - 1
Approach B	Compare coins in pairs	n / 2
Approach C	Divide pile in half, weigh halves	log2(n)

Fake Coin – Approach A (One by One)

Place one coin on the left side, then try each remaining coin on the right.

```
Coin 1 vs Coin 2  →  balanced
Coin 1 vs Coin 3  →  balanced
Coin 1 vs Coin 4  →  balanced
  ...
Coin 1 vs Coin k  →  LIGHTER! (Coin k is fake)
```

- **Best case:** 1 weighing (lucky)
- **Worst case:** n - 1 weighings
- For 1,024 coins: up to **1,023** weighings

Fake Coin – Approach B (Pairs)

Pair up coins and weigh each pair.

```
(Coin 1, Coin 2)  →  balanced (both real)
(Coin 3, Coin 4)  →  balanced (both real)
   ...
(Coin k, Coin k+1) → UNBALANCED! (one of these is fake)
```

- **Worst case:** n / 2 weighings
- For 1,024 coins: up to **512** weighings
- Better, but still linear

Fake Coin – Approach C (Divide in Half)

Split the pile in half, weigh the two halves. The lighter half contains the fake.

```
n = 1024 coins

Step 1:  512 vs 512  →  left is lighter
Step 2:  256 vs 256  →  right is lighter
Step 3:  128 vs 128  →  left is lighter
   ...
Step 10:   1 vs   1  →  left is lighter → FOUND!
```

- **Always** log2(n) weighings
- For 1,024 coins: exactly **10** weighings

This is the **Divide and Conquer** strategy – we'll study it in depth in **Chapter 3**.

Fake Coin – Comparison

For n = 1,024 coins:

Approach	Worst case	Strategy
A (one by one)	**1,023**	Brute force
B (pairs)	**512**	Slightly smarter
C (halving)	**10**	Divide and Conquer

Why is Approach C so much better?

- Each weighing **eliminates half** the remaining coins
- The number of weighings = log2(n)

> The **log function** is crucial in algorithm analysis. Remember it!

1.7 Poisoned Wine Problem

Story: A king has many wine jars. A spy poisoned **exactly one** jar. The poison kills exactly **one week** after tasting, even from a tiny sip.

The king's order:

- Find the poisoned jar in **exactly one week**
- **Minimize** the number of servants who must taste

Poisoned Wine – Small Cases

2 jars, 1 servant:

Servant tastes jar…	If servant dies	If servant lives
Jar 1	Jar 1 is poisoned	Jar 2 is poisoned

1 servant can identify the poisoned jar among **2** jars.

Poisoned Wine – 4 Jars

First attempt: 3 servants each taste 1 jar -> works but uses 3 servants.

Can we do it with 2 servants?

Assign each jar a 2-bit binary number:

Jar	Binary	Servant A tastes?	Servant B tastes?
0	00	No	No
1	01	No	Yes
2	10	Yes	No
3	11	Yes	Yes

Rule: Servant A tastes jars where **bit 1 = 1**. Servant B tastes jars where **bit 0 = 1**.

Poisoned Wine – Reading the Result

After one week, check who died:

A dies?	B dies?	Binary	Poisoned jar
No	No	00	Jar 0
No	Yes	01	Jar 1
Yes	No	10	Jar 2
Yes	Yes	11	Jar 3

The **death pattern** directly encodes the **binary number** of the poisoned jar!

Poisoned Wine – 8 Jars (3 Servants)

Jar	Binary	A tastes?	B tastes?	C tastes?
0	000	No	No	No
1	001	No	No	Yes
2	010	No	Yes	No
3	011	No	Yes	Yes
4	100	Yes	No	No
5	101	Yes	No	Yes
6	110	Yes	Yes	No

Poisoned Wine – General Solution

For n jars:

- Number of servants needed = **log2(n)**
- Each jar is assigned a unique binary number (0 to n-1)
- Servant k tastes all jars where the k-th bit is 1
- After one week, the binary pattern of dead servants = the poisoned jar's number

```
n = 1000 jars  →  log2(1000) = ~10 servants
n = 1,000,000  →  log2(1,000,000) = ~20 servants
```

This is the power of **binary encoding** – a fundamentally different way of thinking.

Summary

Problem	Algorithm / Strategy	Key Concept
Finding max	Scan all elements	Sequential Search
Finding a target	Divide sorted data in half	Binary Search
Coin change	Pick largest coin first	Greedy (Ch. 4)
Euler path	Follow cycles, avoid bridges	Euler Circuit / DFS
Fake coin	Split pile in half	Divide and Conquer (Ch. 3)
Poisoned wine	Binary number assignment	Binary Encoding

Recurring theme: The **log2(n)** function appears in binary search, fake coin, and poisoned wine. Algorithms that reduce the problem size by half at each step achieve **logarithmic** performance.

What's Next

- **Week 02:** Algorithm Analysis – Big-O notation, time complexity, asymptotic analysis
- **Take-home assignment** starts next week
- No quiz and no homework this week (OT week)

Q & A

codingchild@korea.ac.kr